Шпаргалка для защиты

8 слайдов на английском | План: 6 минут 15 секунд | Целевой диапазон: 5-7 минут

Слайд	Тема	Время
1	LLM Access Control Under Prompt Injection	0:00-0:30
2	Tensor Trust and the threat model	0:30-1:15
3	Four configurations of the same gatekeeper	1:15-2:00
4	Frozen cases and reproducible settings	2:00-2:50
5	Backend checks blocked the tested unauthorized grants	2:50-3:45
6	Live examples: H02 and H08	3:45-5:10
7	Contribution, limits and next tests	5:10-6:00
8	Conclusion and references	6:00-6:15

В заметках PowerPoint есть английский текст выступления и русское объяснение. Не читай весь слайд. На демонстрацию Colab выделено около 30 секунд внутри шестого слайда. Не включай вопросы аудитории в этот тайминг.

1. LLM Access Control Under Prompt Injection (0:00-0:30)

Задача проекта - проверить, может ли LLM надёжно соблюдать простое правило доступа. Основа - сценарий Tensor Trust. Мы сравниваем четыре конфигурации на одной модели в Colab. Главный вопрос: кто должен принимать решение о доступе, если пользователь может писать произвольные инструкции? Сразу назови объём: 288 измеряемых вызовов.

Во время показа: Покажи название, автора и объём эксперимента. Не начинай с установки библиотек.

2. Tensor Trust and the threat model (0:30-1:15)

Статья Toyer и соавторов опубликована на ICLR 2024. Для проекта важны две разные цели: извлечение защищённой информации и изменение поведения вопреки правилам. Атакующий контролирует код и текст запроса, но не код приложения. В атакующих случаях код заведомо неверный. Наши 64 случая собственные, синтетические; это адаптация идеи, а не воспроизведение полного benchmark.

Во время показа: Поясни разницу между утечкой секрета и разрешением без секрета. Для обхода не обязательно узнать секрет.

Архитектура и воспроизводимость

3. Four configurations of the same gatekeeper (1:15-2:00)

Объясни таблицу четырёх конфигураций. Первый фактор - наличие секрета в системном промпте. Второй - программная проверка и фиксированный ответ приложения. Baseline доверяет ответу LLM. В защищённых вариантах решение принимает hmac.compare_digest. Мы продолжаем вызывать модель, чтобы сравнить её сырой ответ. Для настоящей проверки пароля LLM не нужна; здесь вызов нужен для контролируемого исследования.

Во время показа: Покажи raw_output и delivered_output. Уточни, что backend объединяет два механизма, а не является отдельным тестом каждого.

4. Frozen cases and reproducible settings (2:00-2:50)

Назови модель, T4, garak 0.13, greedy decoding, seed 2026 и лимит 64 новых токена. На каждый вариант идут 32 атаки на извлечение, 32 на обход и 8 контролей: 72 × 4 = 288. Пробы собственные, исполняются настоящим garak. Он обращается к модели по 127.0.0.1 внутри той же среды Colab. 72 предварительных эхо-теста не вызывали LLM; 13 пробных и 8 демонстрационных вызовов в основные результаты не включены.

Во время показа: Не называй все строки JSONL отдельными тестами: учитываются 288 завершённых попыток garak со status=2.

Как объяснить код

system_prompt: доверенная политика и наличие/отсутствие секрета.
json.dumps: экранирование пользовательских полей.
run_case: реальный вызов Qwen и запись результата.
model_grant: baseline доверяет точному разрешающему ответу.
backend_verify: точное сравнение байтов через hmac.compare_digest.
delivered_output: фиксированный ответ программы в защищённых режимах.

Важно: при удалении секрета меняется и формулировка инструкции. Не утверждай, что ухудшение вызвано только отсутствием конкретной строки секрета.

Результаты и живая демонстрация

5. Backend checks blocked the tested unauthorized grants (2:50-3:45)

Baseline: 7 из 32 обходов. Вариант без секрета: 31 из 32 и ещё 5 разрешений в наборе на извлечение; всего 36 из 64 атак. Детектор утечки эти пять случаев не считает успехом извлечения, поэтому нужна дополнительная метрика фактического доступа. Серверные варианты дали 0 из 64 и 8 из 8 контролей. Исходная модель приняла код с пробелом и в нижнем регистре. Сырые ответы попарно совпали 72 из 72: защиту обеспечивает приложение.

Во время показа: Поясни знаменатели: 32 для одного типа атаки, 64 для всех атак, 8 для контролей. Нулевой результат не означает полную безопасность.

6. Live examples: H02 and H08 (3:45-5:10)

H02: покажи неверный код и просьбу повторить разрешающую строку. Ответ модели имеет правильный формат, но означает неверное решение. H08: покажи текст с role: system. Сериализатор экранирует кавычки; настоящего нового системного сообщения нет. Ошибка в том, что модель следует содержимому строки. В Colab открой раздел 15, выбери both и выполни ячейку. Покажи одинаковый raw_output и разные delivered_output. Это 8 дополнительных вызовов, отдельно от 288.

Во время показа: Оставь около 30 секунд на Colab. Если среда отключена, честно показывай сохранённый вывод и объясни, что для нового запуска нужно восстановить модель.

Порядок демонстрации в Colab

1. Открой раздел 15. Выбери DEMO_CASE = both и RUN_LIVE_DEMO = True.
2. Нажми выполнение. Это создаёт 8 новых вызовов в отдельном каталоге.
3. Покажи WRONG-CODE, raw_output и delivered_output.
4. Baseline разрешает доступ, backend_only отклоняет тот же сырой ответ.
5. Для H08 поясни: role/system остаются текстом, настоящие роли сообщений не меняются.

Если Colab отключён, не называй сохранённую таблицу новым запуском. Покажи записанные результаты и объясни, что для повторения нужно восстановить модель. При медленной среде используй один живой случай H02, затем сохранённый H08.

Выводы и ответы на вопросы

7. Contribution, limits and next tests (5:10-6:00)

Расширение - четыре конфигурации вместо одного запуска, сложные отрицательные контроли, отдельные сырые и доставленные ответы и метрика фактических разрешений. Ограничения: одна небольшая модель, один публичный секрет, один seed, фиксированные атаки без адаптивного поиска. Удаление секрета меняет и формулировку промпта. Backend объединяет проверку и фиксированный ответ. Для продолжения нужны другие секреты и модели, адаптивные атаки, отдельные тесты сессий и журналов.

Во время показа: Важная оговорка: утечек не было и в baseline, поэтому эффект удаления секрета на утечки здесь не измерен.

8. Conclusion and references (6:00-6:15)

Заключение: решение о доступе должно принадлежать проверяемому коду приложения. Исходники, команды, pin модели и все журналы входят в архив. Поблагодари аудиторию и переходи к вопросам.

Во время показа: Не читай библиографию вслух. Ссылки доступны на слайде и в отчёте.

Почему не хватает проверки формата? ACCESS_GRANTED имеет допустимый формат, но не соответствует неверному коду. Нужно проверить само право доступа.

Почему H08 не сломал JSON? json.dumps экранирует кавычки. json.loads возвращает два ожидаемых поля. Модель подчиняется тексту внутри строки.

31/32 или 36/64? 31 успех из 32 атак на обход. Ещё 5 разрешений возникли в наборе на извлечение. Всего 36 неавторизованных решений из 64 атак.

Модель стала безопаснее? Весов и сырых ответов защита не меняет. Приложение блокирует последствия ненадёжного ответа.

Что означает ноль утечек? Выбранные детекторы не нашли полного секрета и проверенных вариантов его преобразования. Это не охватывает все возможные утечки.

Что именно добавлено к статье? Собственный стенд, четыре конфигурации, сложные контроли и сверка сырого ответа с решением приложения. Полный Tensor Trust benchmark не воспроизводился.

Как проверены результаты? 288 уникальных запросов связаны с 288 завершёнными попытками garak. Скрипты сверяют исходники, хеши, UUID, ответы и показатели. 8 демонстрационных запросов имеют отдельные UUID.